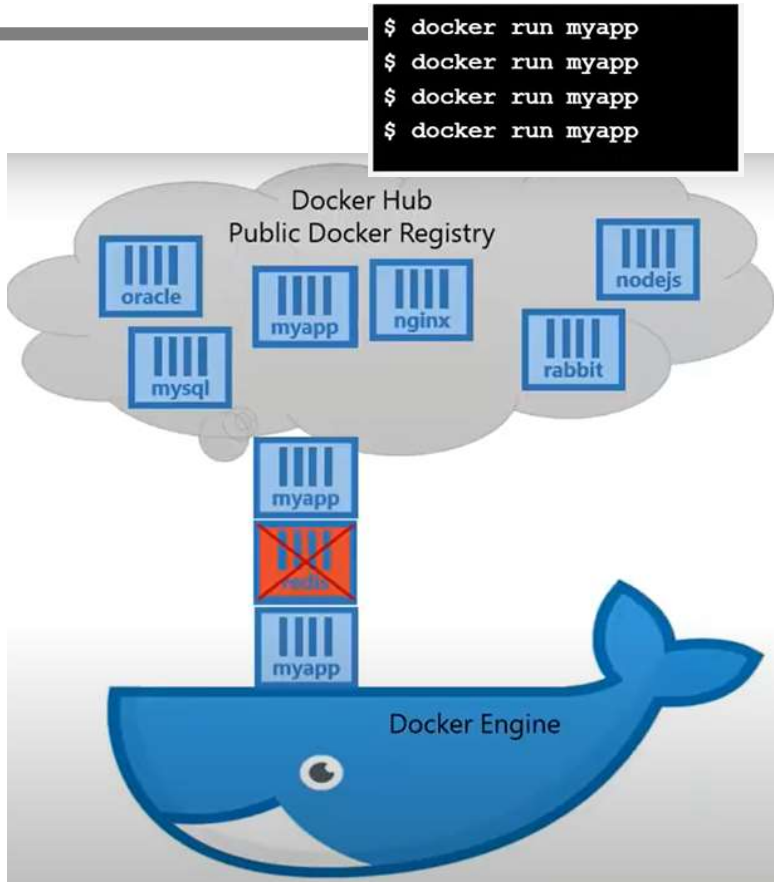
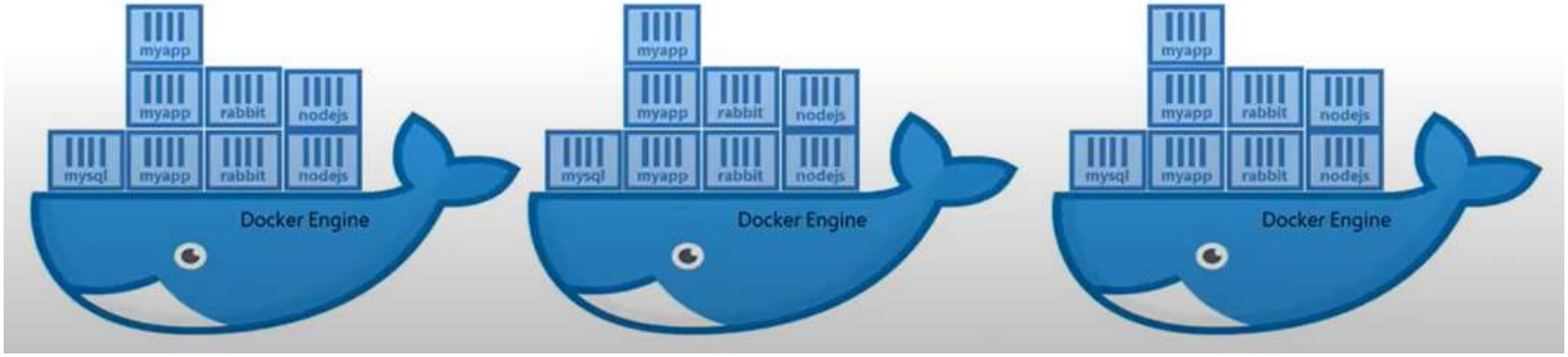
Orchestration des conteneurs

- Besoin de démarrer plusieurs instances pour des raisons de:
  - **scalabilité**: répondre de manière efficace à une variation de la charge de travail
  - **tolérance aux pannes**: capacité à continuer de fonctionner de manière fiable et à fournir des services même en présence de pannes matérielles, logicielles ou de réseau
  - **disponibilité**: système est fonctionnel et répond aux demandes des utilisateurs de manière fiable et sans interruption pendant une période donnée
  - **monté en charge**: augmentation de la quantité de travail sur le système
- Si une instance tombe en panne, il faut la remplacer
- Si Docker Engine tombe en panne, tous les conteneurs vont cracher.
- L'**orchestration** des conteneurs est une solution pour ce problème



Orchestration des conteneurs

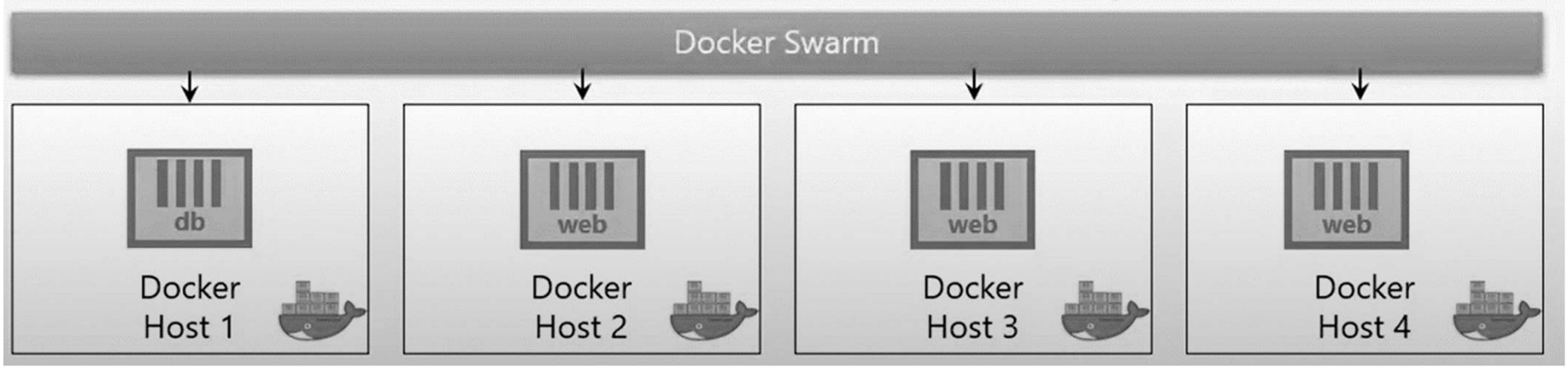
- Consiste en un ensemble d'outils et de scripts qui permettent l'automatisation des opérations:
  - Scalabilité des conteneurs (capacité d'adapter la taille d'un ensemble de conteneurs en fonction des besoins), réplication des conteneurs, haute disponibilité, bilan de santé des conteneurs etc.
- Exemples d'outils:
  - Docker Swarm
  - Kubernetes
  - MESOS



Orchestration des conteneurs

Docker Swarm

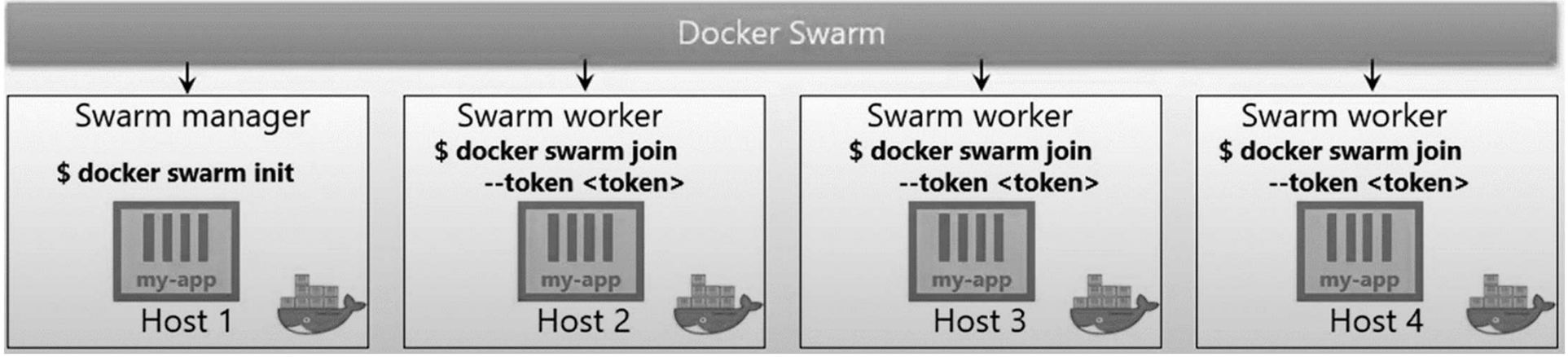
- Docker Swarm permet de combiner plusieurs machines ensemble pour former un cluster.
- Il prend en charge la distribution et la réplication des instances de services des applications dans des hosts distribués pour la haute disponibilité et l'équilibrage de charges à travers différents systèmes et hardwares.



Orchestration des conteneurs

Configuration de Docker Swarm

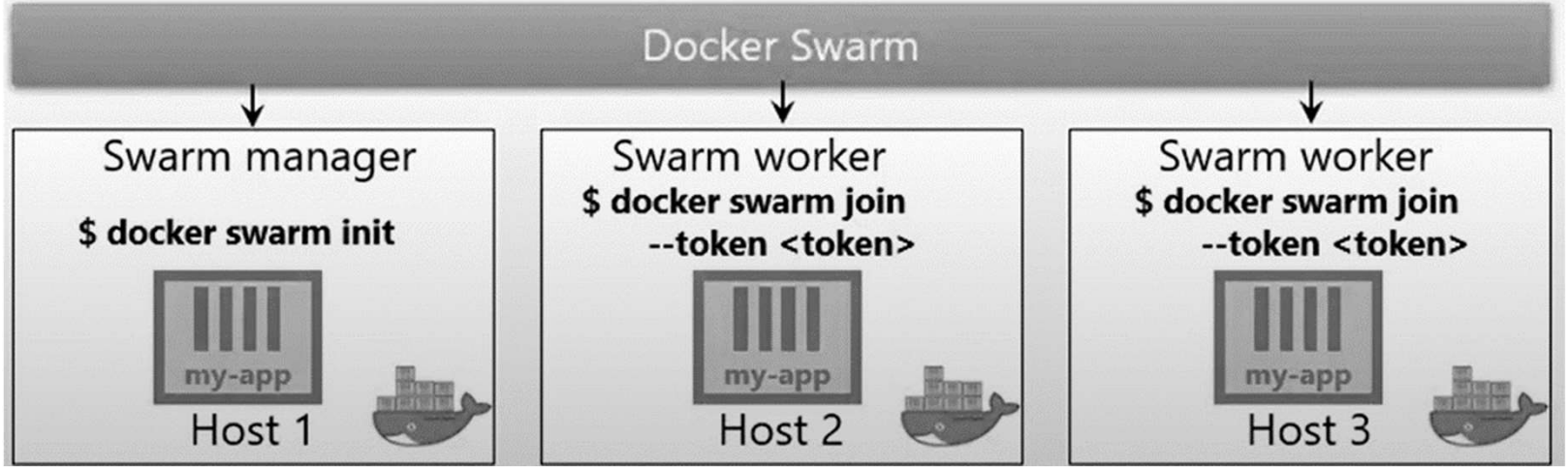
- Pour configurer Swarm il faut:
  - Démarrer plusieurs hosts docker
  - Désigner un host en tant que **manager** et les autres en tant que **workers**
  - Par la suite exécuter les commandes "docker service" sur le swarm manager node
    - $ docker service create --replicate=4 -p 8080:80 may-app



Orchestration des conteneurs

Exemple d'application Swarm

- Démarrer 3 VM ubuntu avec Docker engine installé:
  - Host 1 avec l'adresse IP 192.168.57.3
  - Host 2 avec l'adresse IP 192.168.57.4
  - Host 3 avec l'adresse IP 192.168.57.5
- Ensuite dans une machine (Machine Physique), connecter 3 terminaux avec SSH au 3 hosts

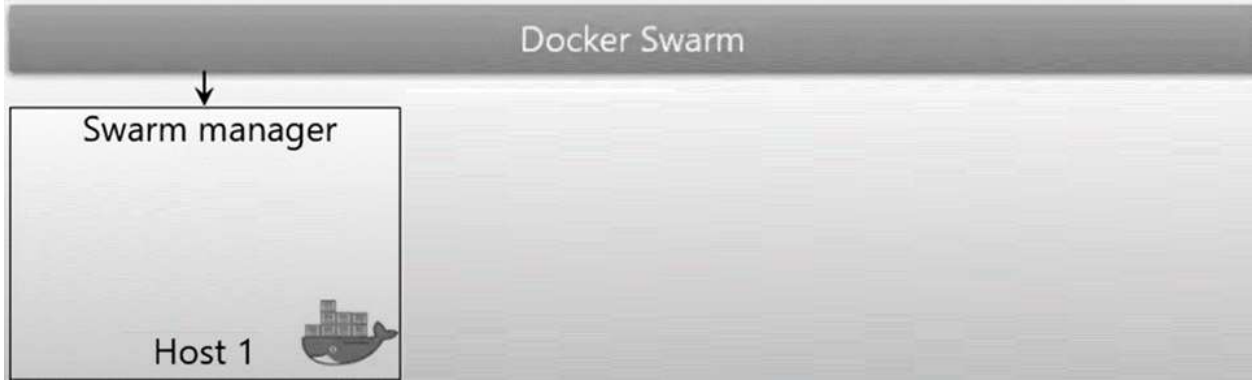


Orchestration des conteneurs

Démarrer un Swarm Manager

```
$ sudo docker swarm init --advertise-addr 192.168.57.3 --listen-addr 192.168.57.3
Swarm initialized: current node (ssslatiguj2krc73fv7l43r7l) is now a manager.
To add a worker to this swarm, run the following command:
    docker swarm join --token SWMTKN-1-3w4cwlyti2kelxf70vb2m0ul56rbavtsrnwnoc7s3
9fi7pdjzj-3w7zsj6go18iybcm7reoha7e4 192.168.57.3:2377
To add a manager to this swarm, run 'docker swarm join-token manager' and follow the
instructions.
```

Advertise-addr: adresse où swarm manager accepte les connexions à partir des autres nœuds
Listen-addr: adresse d'écoute pour accéder au cluster swarm depuis l'extérieur
Le port par défaut de Swarm est 2377

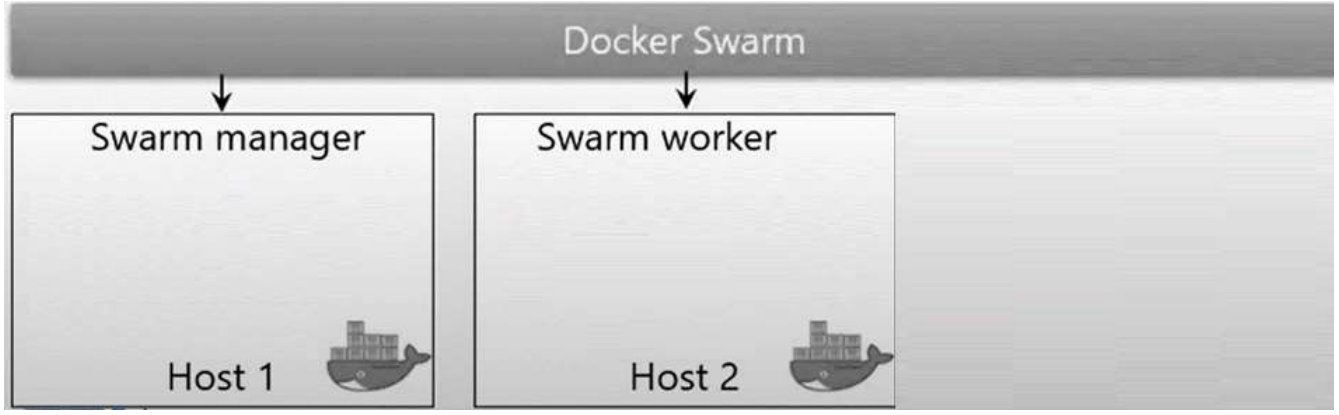


Orchestration des conteneurs

Démarrer un nœud Swarm Worker

```
$ sudo docker swarm join --token SWMTKN-1-
3w4cwlyti2kelxf70vb2m0ul56rbavtsrnwnoc7s39fi7pdjzj-3w7zsj6go18iybcm7reoha7e4
192.168.57.3:2377

This node joined a swarm as a worker.
```



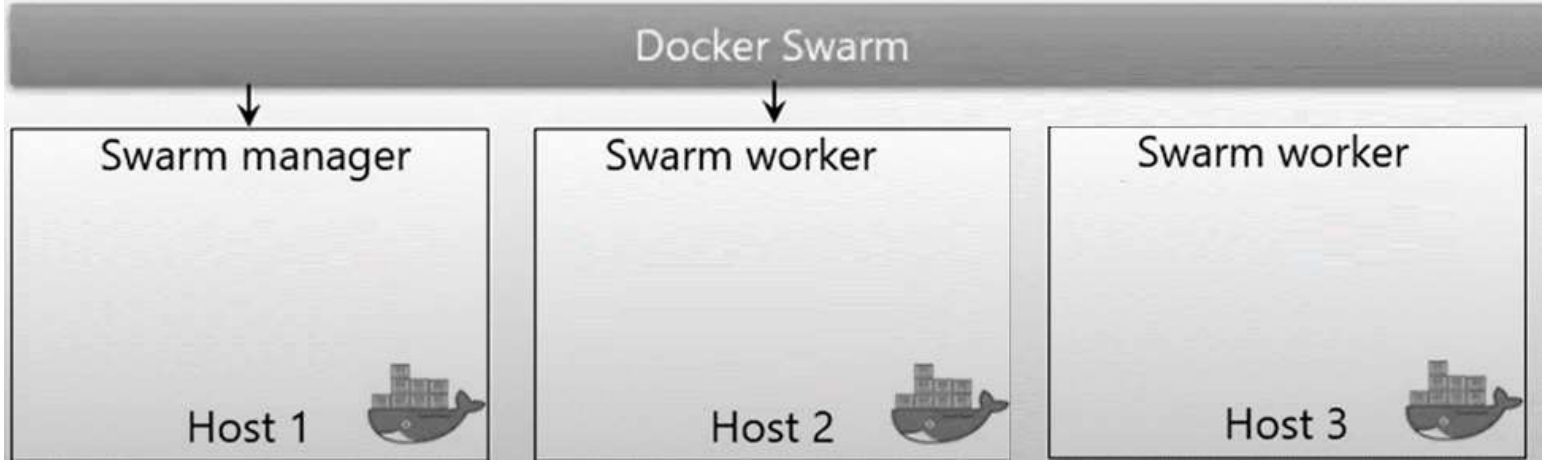
Orchestration des conteneurs

Démarrer un nœud Swarm Worker

On refait la même chose sur une autre machine

```
$ sudo docker swarm join --token SWMTKN-1-
3w4cwlyti2kelxf70vb2m0ul56rbavtsrnwnoc7s39fi7pdjzj-3w7zsj6go18iybcm7reoha7e4
192.168.57.3:2377

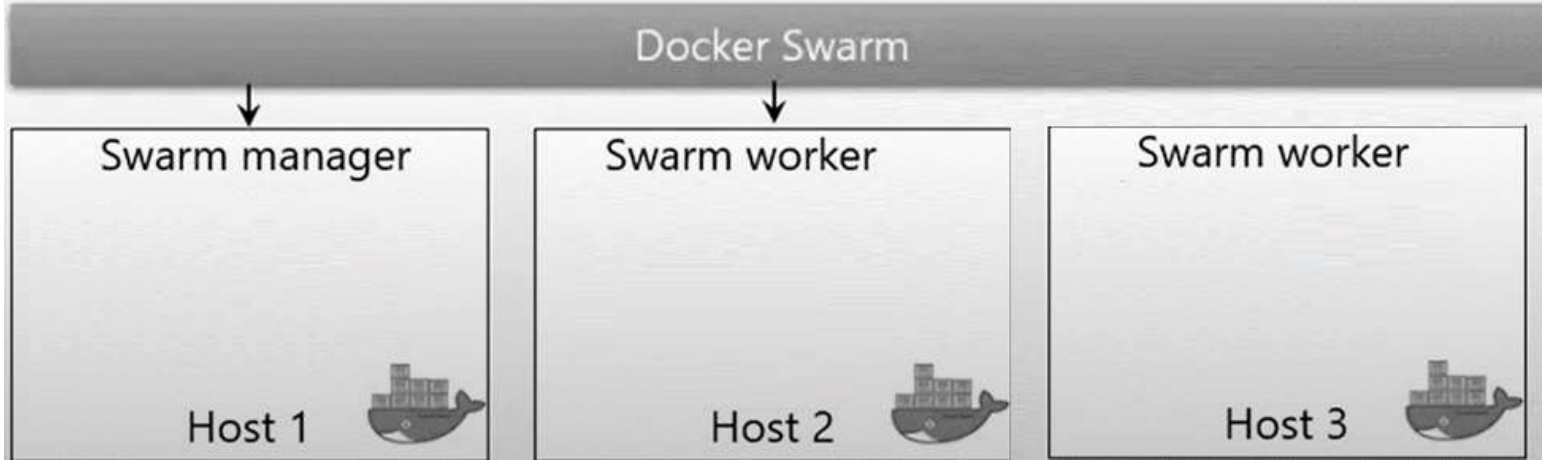
This node joined a swarm as a worker.
```



Orchestration des conteneurs

Lister les nœuds du cluster Swarm

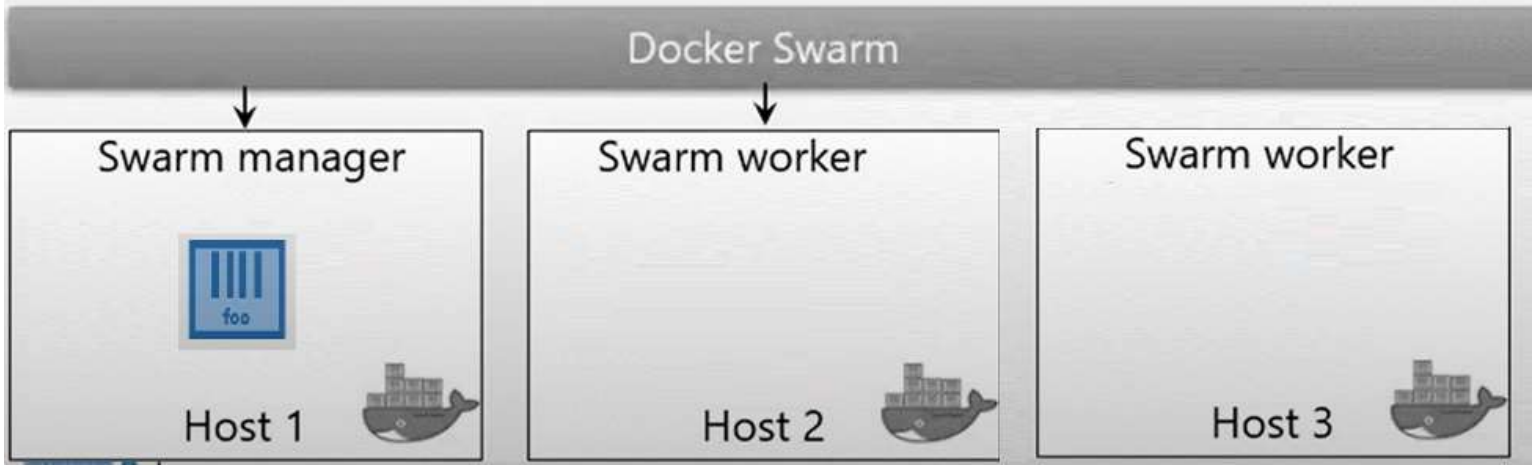
```
$ sudo docker node ls
ID                            HOSTNAME  STATUS  AVAILABILI  TY  MANAGER STATUS  ENGINE VERSION
ssslatiguj2krc73fv7143r7l *   host1     Ready   Active          Leader          19.03.1
tw9is2zomef5zya9b9w7wp37k     host2     Ready   Active                          19.03.1
graghbpp1oymjdjn2bfdp2s7i     host3     Ready   Active                          19.03.1
```



Orchestration des conteneurs

Créer et démarrer un service Swarm

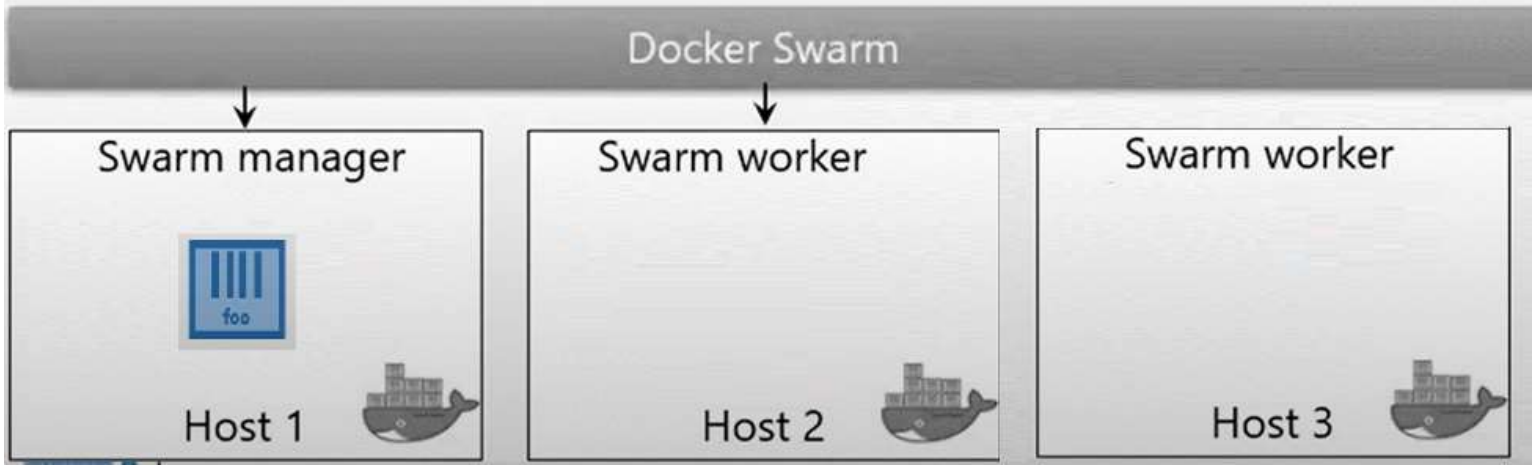
```
$ sudo docker service create --name foo alpine ping 8.8.8.8
i4j4cvvg250jkfx96ge8l1xhg
overall progress: 1 out of 1 tasks
1/1: running
verify: Service converged
```



Orchestration des conteneurs

Lister les services démarrés

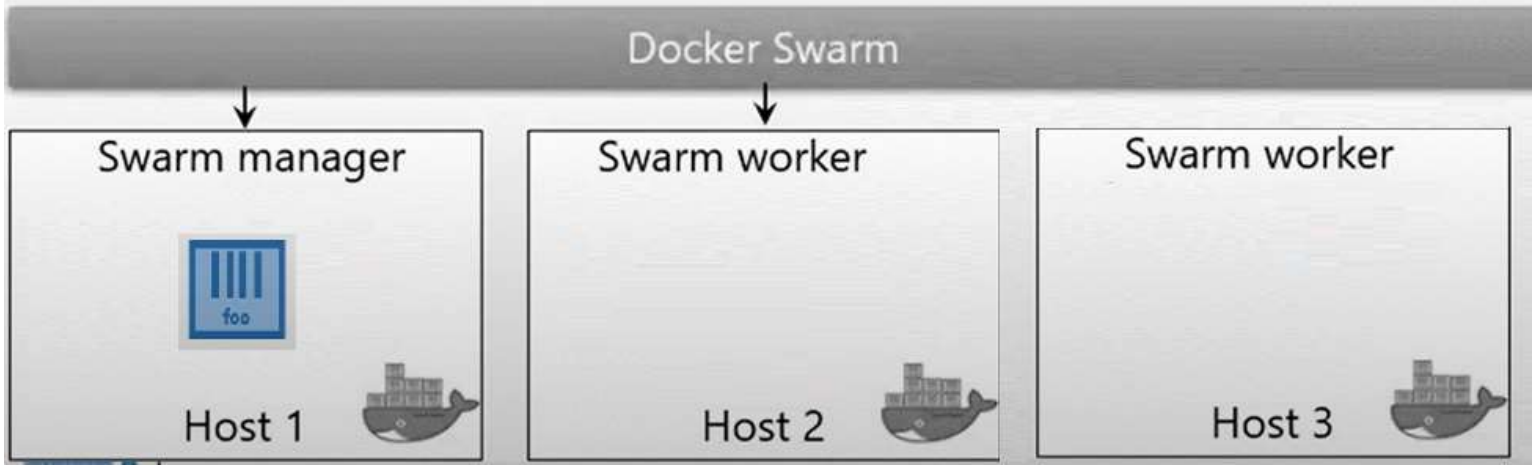
```
$ sudo docker service ls
ID                  NAME                MODE                REPLICAS
IMAGE               PORTS
i4j4cvvg250j        foo                 replicated          1/1
alpine:latest
```



Orchestration des conteneurs

Détails des services démarrés dans Swarm

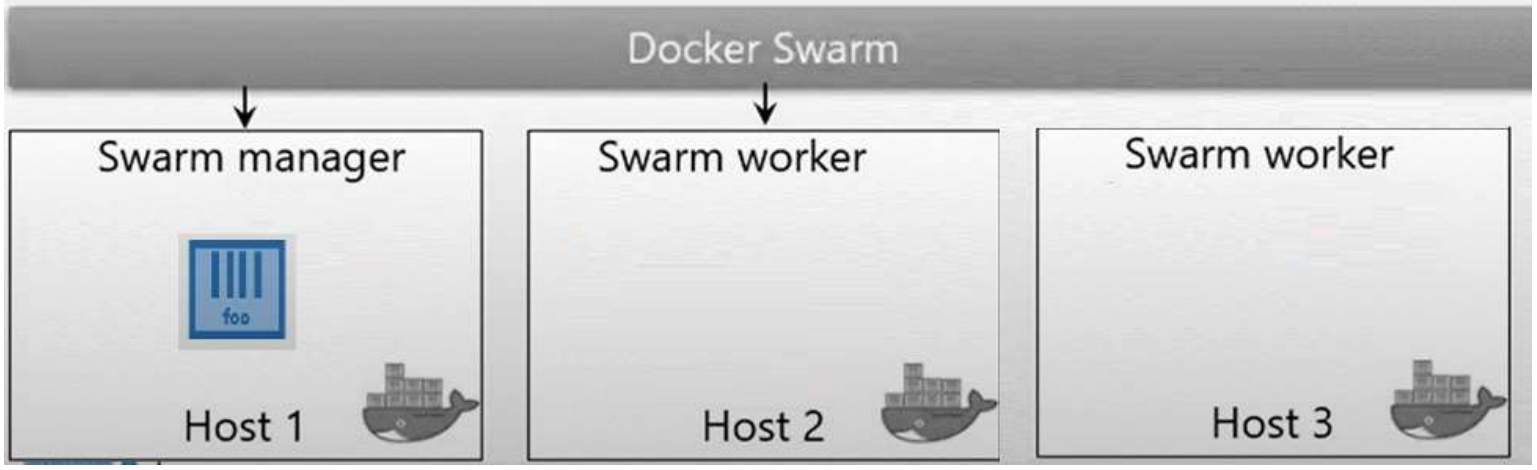
```
$ sudo docker service ps foo
ID            NAME      IMAGE          NODE      DESIRED STATE    CURRENT STATE                ERROR     PORTS
asgwvidbjztj  foo.1     alpine:latest  host1     Running          Running about a minute ago
```



Orchestration des conteneurs

Consulter le fichier logs d'un service

```
$ $ sudo docker service logs foo
foo.1.asgwvidbjztj@host1    | PING 8.8.8.8 (8.8.8.8): 56 data bytes
foo.1.asgwvidbjztj@host1    | 64 bytes from 8.8.8.8: seq=0 ttl=53 time=26.57 ms
foo.1.asgwvidbjztj@host1    | 64 bytes from 8.8.8.8: seq=1 ttl=53 time=9.979 ms
foo.1.asgwvidbjztj@host1    | 64 bytes from 8.8.8.8: seq=2 ttl=53 time=12.075 ms
...
```

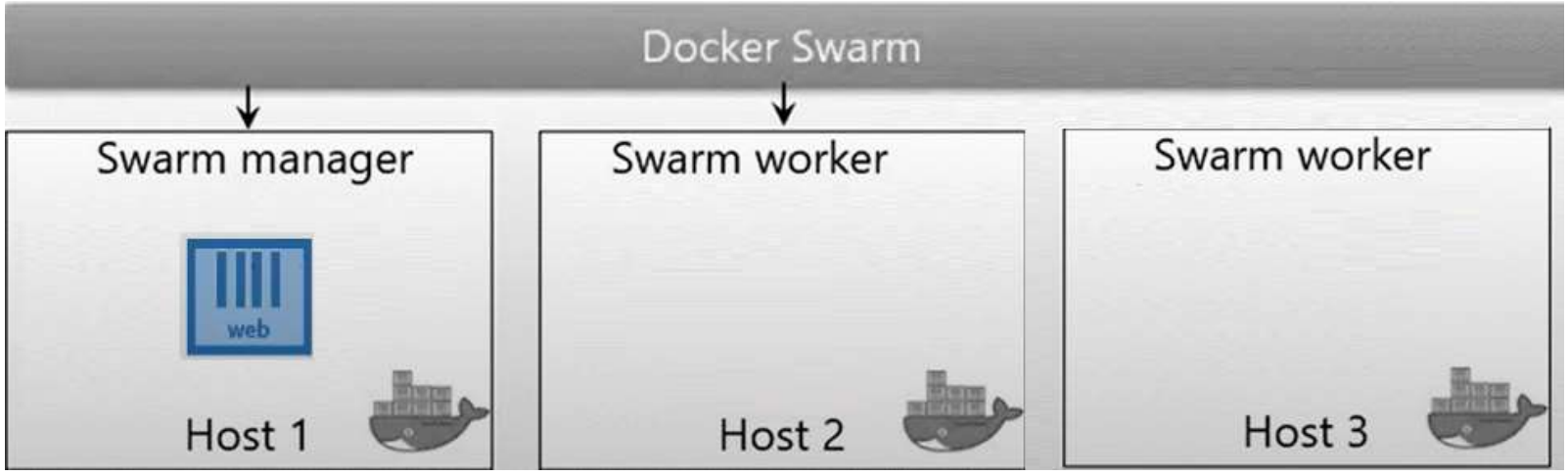


Orchestration des conteneurs

Créer un service: exemple d'application web

```
$ sudo docker service create --name web -p 80:80 emilevauge/whoami
k2xn8uwyr9lgb940krlw5pp15
overall progress: 1 out of 1 tasks
1/1: running
verify: Service converged
```

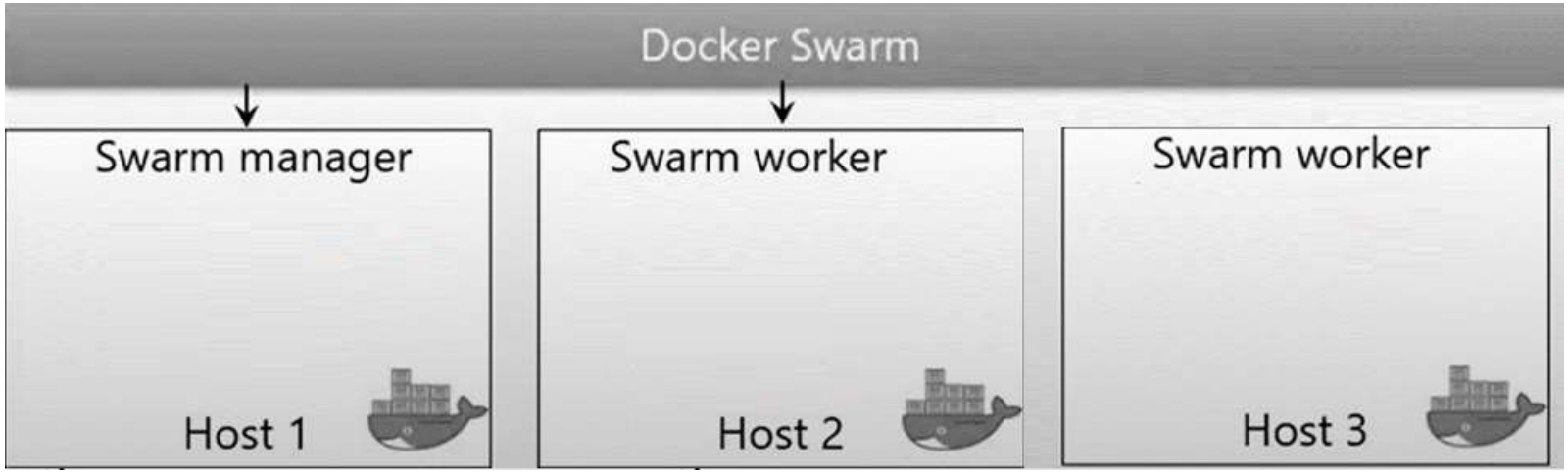
```
$ sudo docker service ps web
ID            NAME      IMAGE                    NODE      DESIRED STATE    CURRENT STATE            ERROR     PORTS
d36tyft9qr07  web.1     emilevauge/whoami:latest  host1     Running          Running 19 seconds ago
```



Orchestration des conteneurs

Supprimer le service web créé

```
$ sudo docker service rm web
web
```

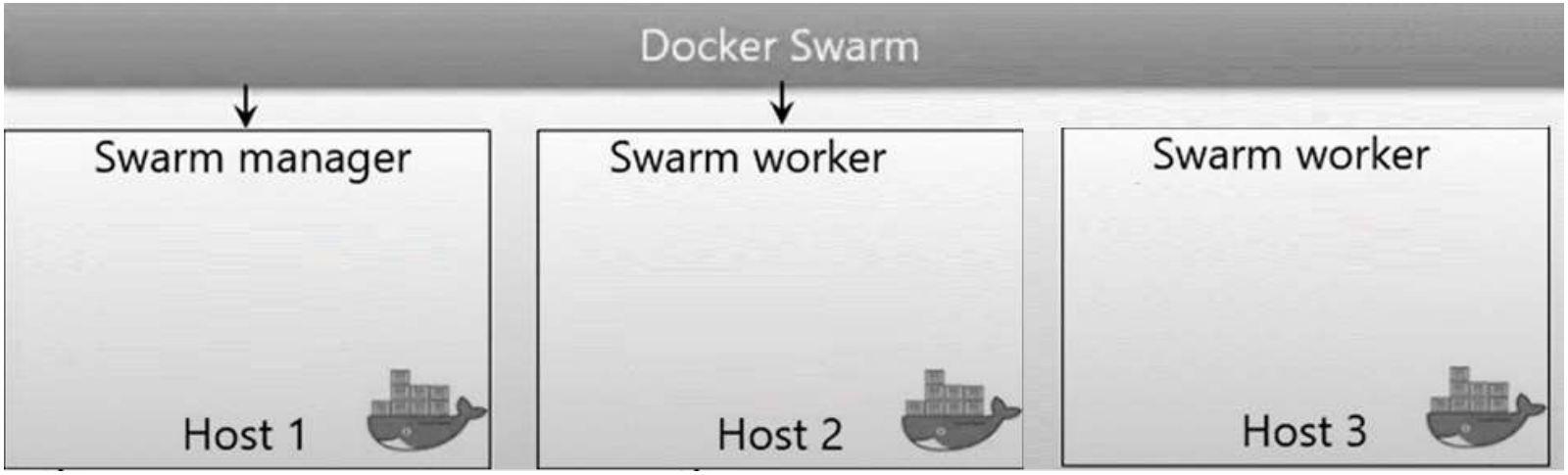


Orchestration des conteneurs

Déployer une application avec trois services mongo, back et front: network sécurisé

```
$ sudo docker network create --driver overlay --opt encrupted
exquisite
zww262r0ze6zzrb67qqz5v0do
```

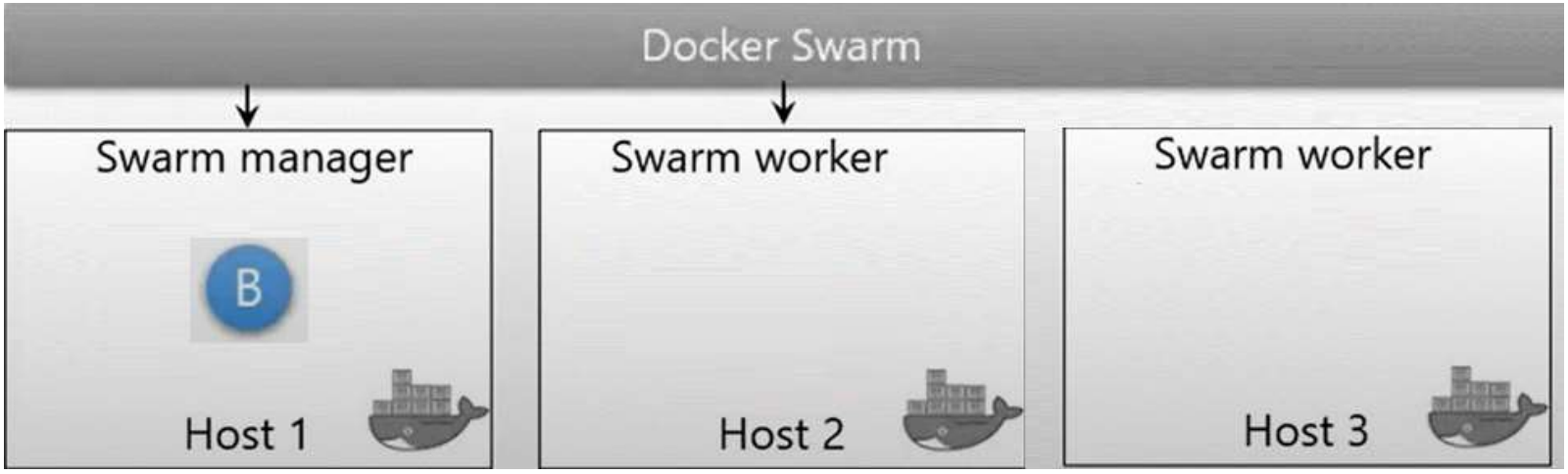
--driver overlay : Les réseaux overlay sont utilisés dans des environnements distribués comme Docker Swarm pour permettre la communication entre les conteneurs sur différents hôtes.



Orchestration des conteneurs

Déployer une application avec trois services mongo, back et front: network sécurisé

```
$  sudo docker service create --name back --network exquisite --limit-memory 64M --reserve-memory 64M vdemeester/exquisite-words-java:v1
pz26k7fz70ok1h48yuu0dwxt5
overall progress: 1 out of 1 tasks
1/1: running
verify: Service converged
```

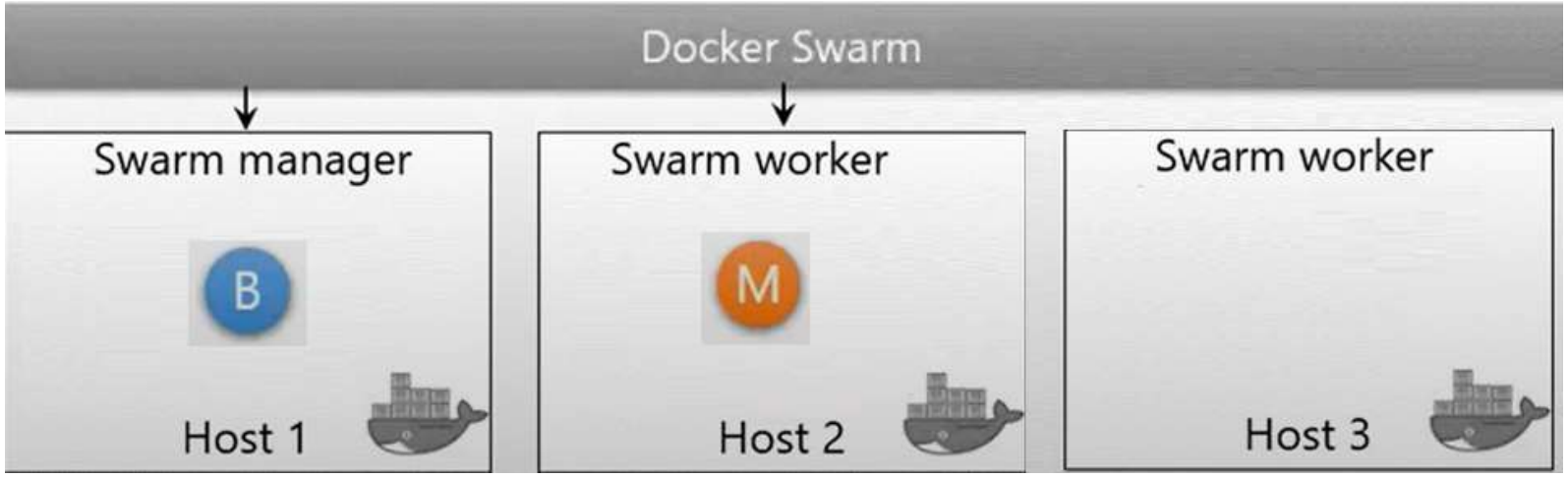


Orchestration des conteneurs

Déployer une application avec trois services mongo, back et front: network sécurisé

```
$ sudo docker service create --name mongo --network exquisite mongo:3.3.8

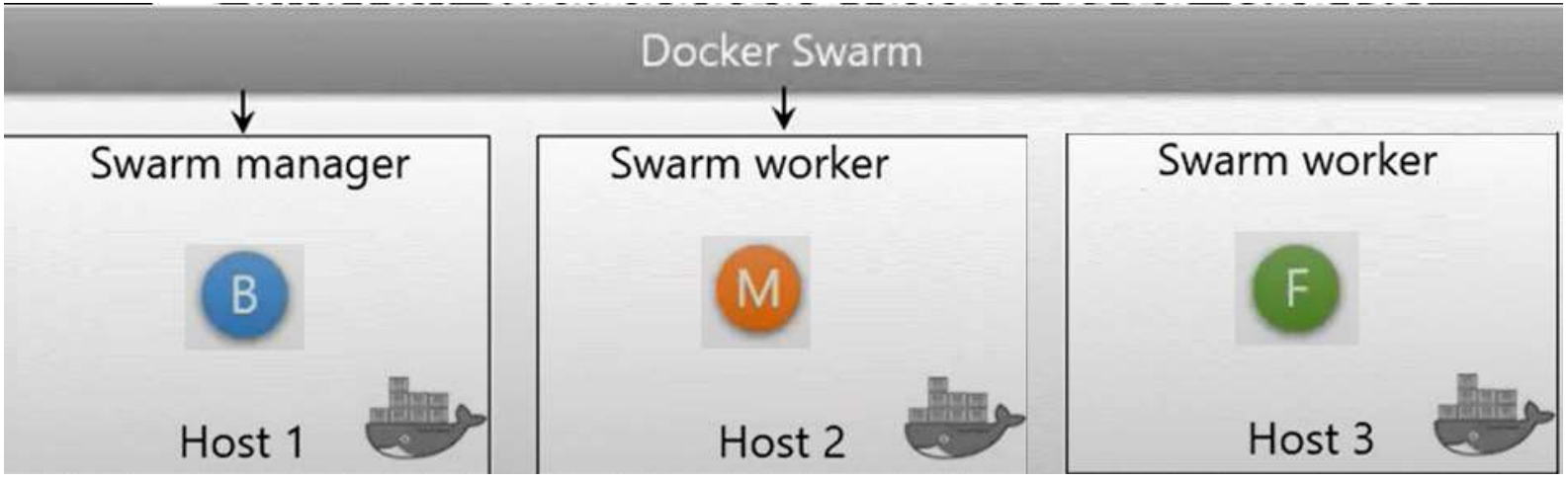
x6vqms9ybjv1fw03m1gheqr1d
overall progress: 1 out of 1 tasks
1/1: running
verify: Service converged
```



Orchestration des conteneurs

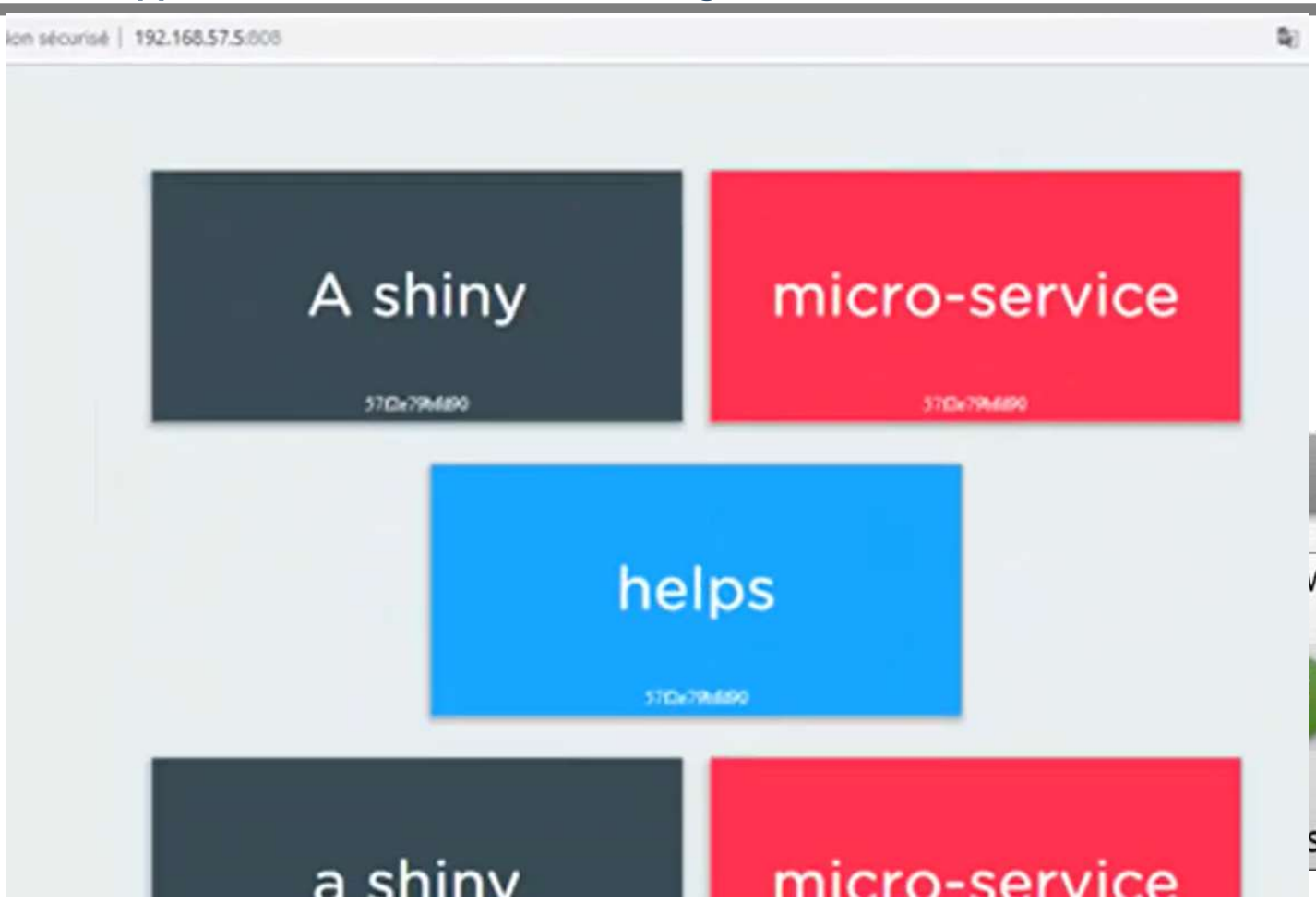
Déployer une application avec trois services mongo, back et front: network sécurisé

```
$ sudo docker service create --name front --network exquisite -p
808:80 vdemeester/exquisite-web:v1
kdttw33h0fn00uq4hksxpny5e
overall progress: 1 out of 1 tasks
1/1: running
verify: Service converged
```



Orchestration des conteneurs

Déployer une application avec trois services mongo, back et front: network sécurisé



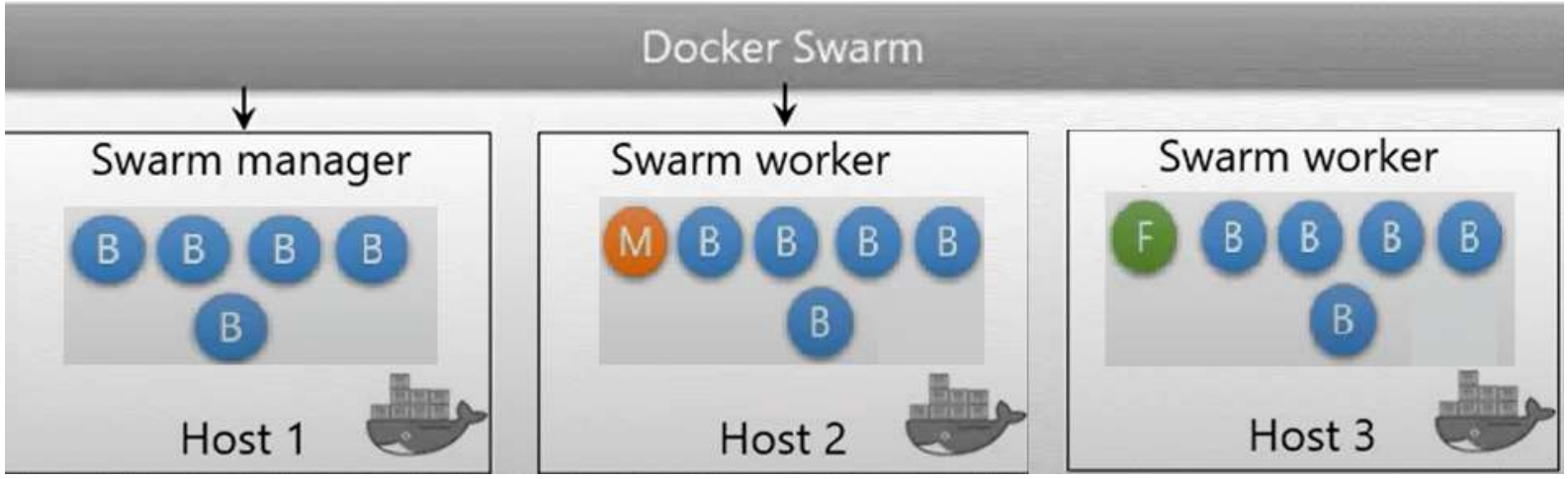


Orchestration des conteneurs

Mise à l'échelle d'un service (scaling back service: 15 instances)

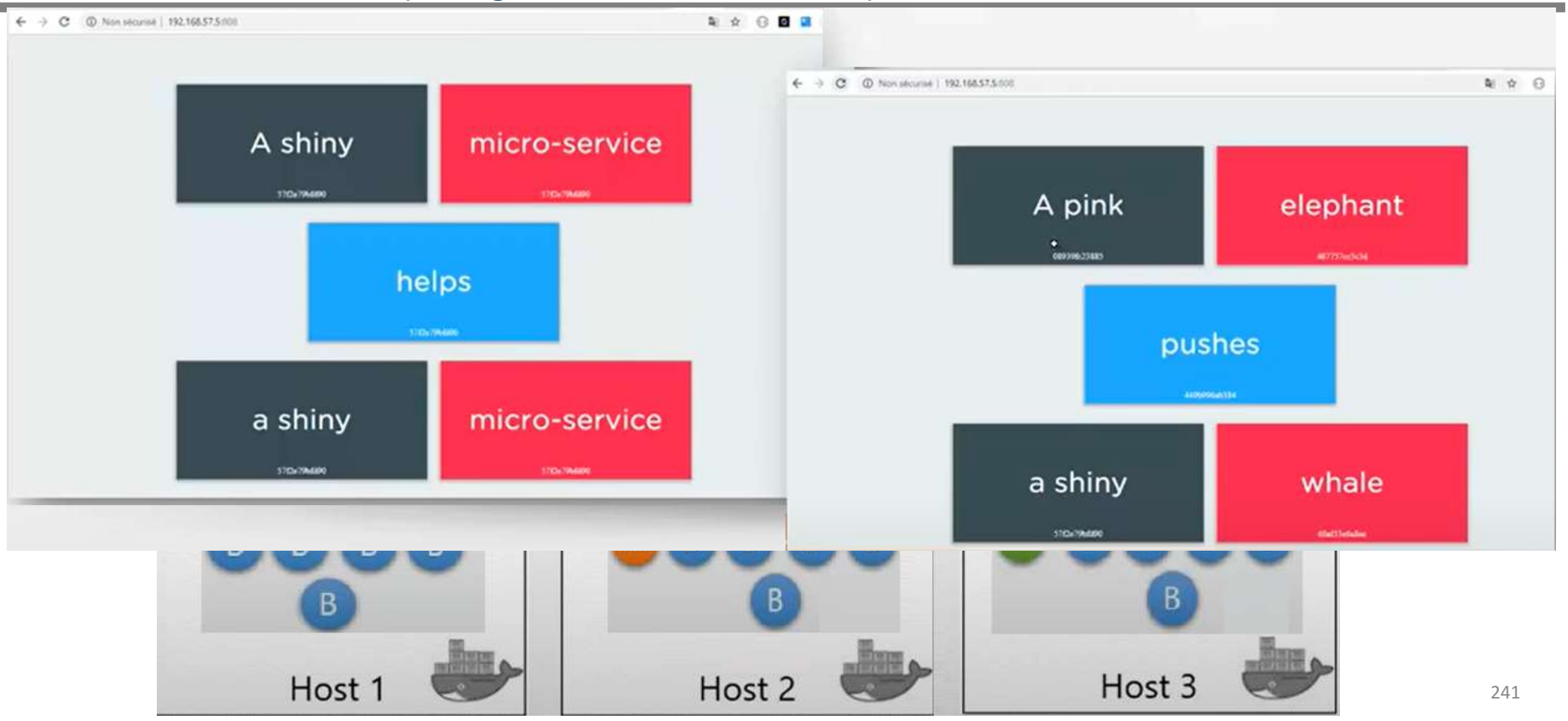
```
$ sudo docker service scale back=15
back scaled to 15
overall progress: 0 out of 15 tasks
1/15:
2/15:
...
9/15: running
15/15: running
verify: Service converged
```

```
$ sudo docker service ls
ID                  NAME                MODE                REPLICAS            IMAGE                                PORTS
pz26k7fz70ok        back                replicated          15/15               vdemeester/exquisite-words-java:v1
kdttw33h0fn0        front               replicated          1/1                 vdemeester/exquisite-web:v1          *:808->80/tcp
x6vqms9ybjvl        mongo               replicated          1/1                 mongo:3.3.8
$ sudo docker service ps back
```



Orchestration des conteneurs

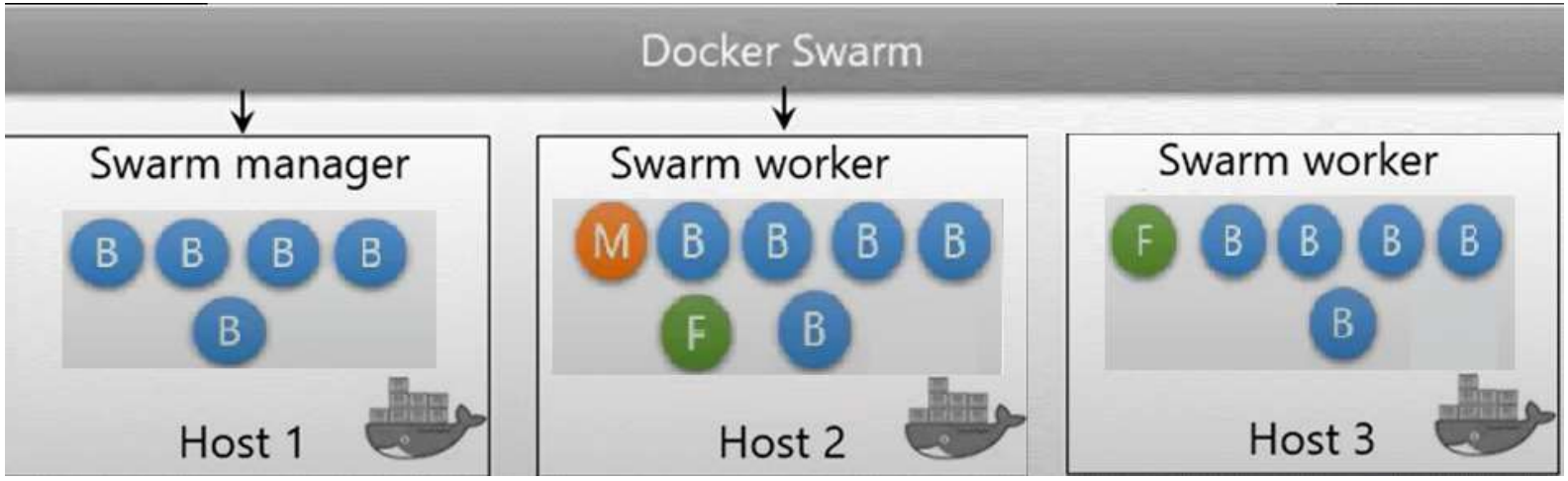
Mise à l'échelle d'un service (scaling back service: 15 instances)



Orchestration des conteneurs

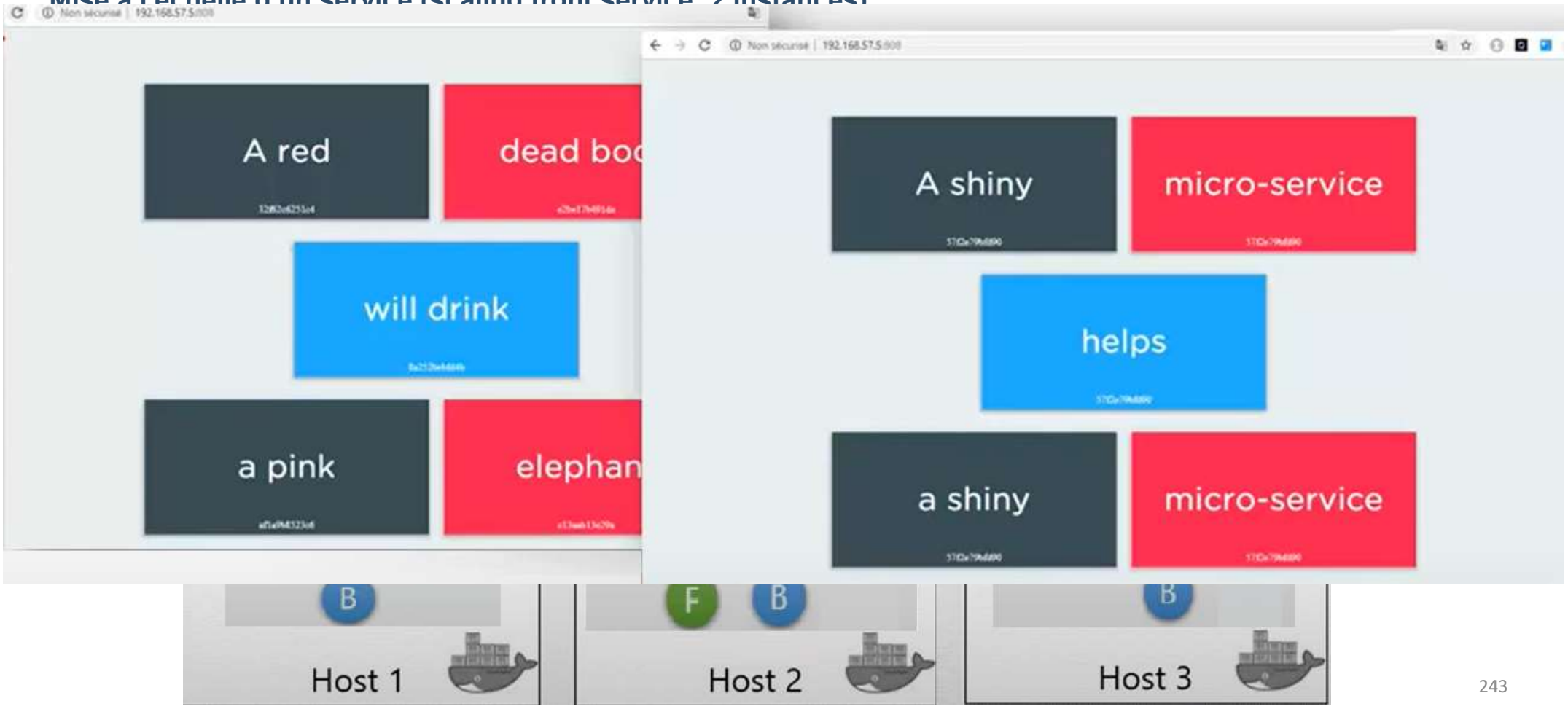
Mise à l'échelle d'un service (scaling front service: 2 instances)

```
$ sudo docker service scale front=2
front scaled to 2
overall progress: 2 out of 2 tasks
1/2: running
2/2: running
verify: Service converged
```



Orchestration des conteneurs

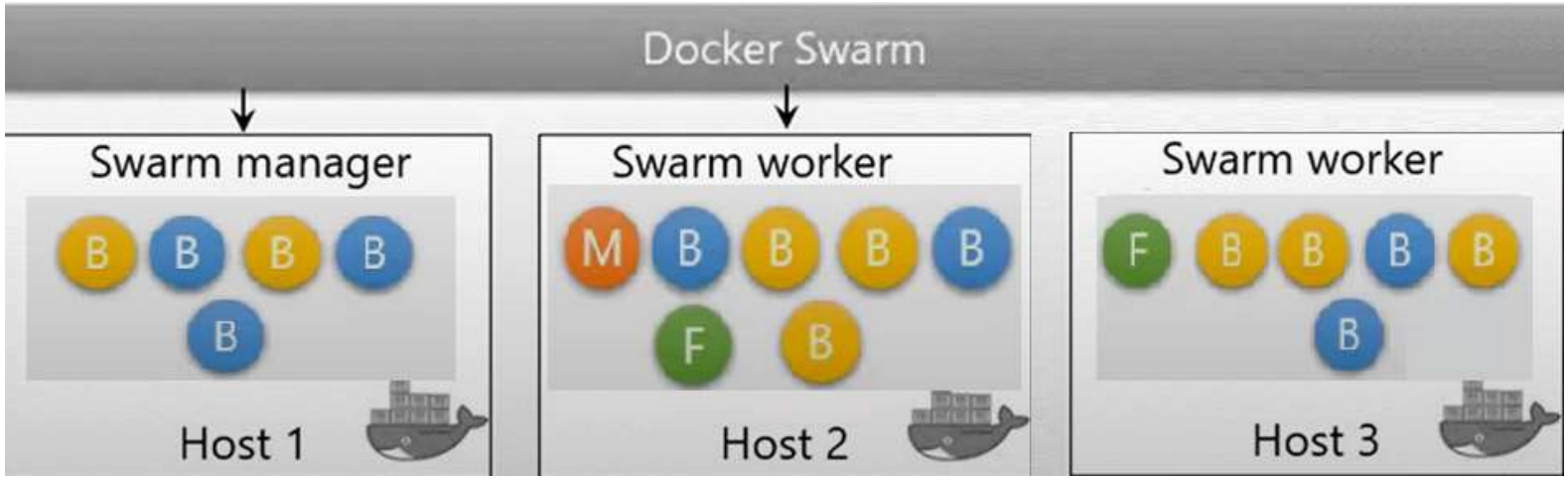
Mise à l'échelle d'un service (scaling front service: 2 instances)



Orchestration des conteneurs

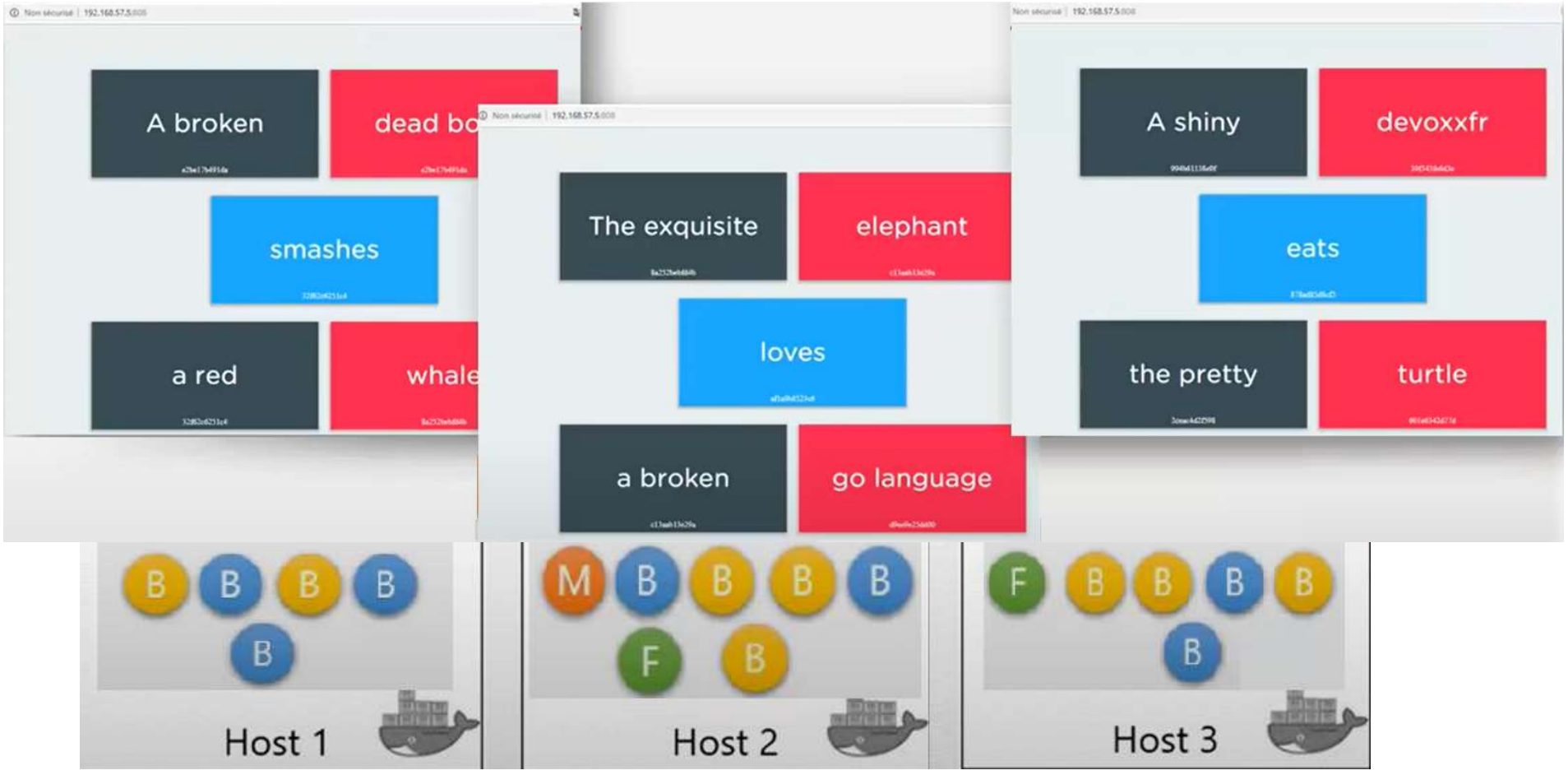
Déployer la mise à jour du service back

```
$ sudo docker service update --update-parallelism 2 --update-delay 10s back
back
overall progress: 0 out of 15 tasks
1/15:
2/15:
...
4/15: running
15/15: running
verify: Service converged
```



Orchestration des conteneurs

Déployer la mise à jour du service back

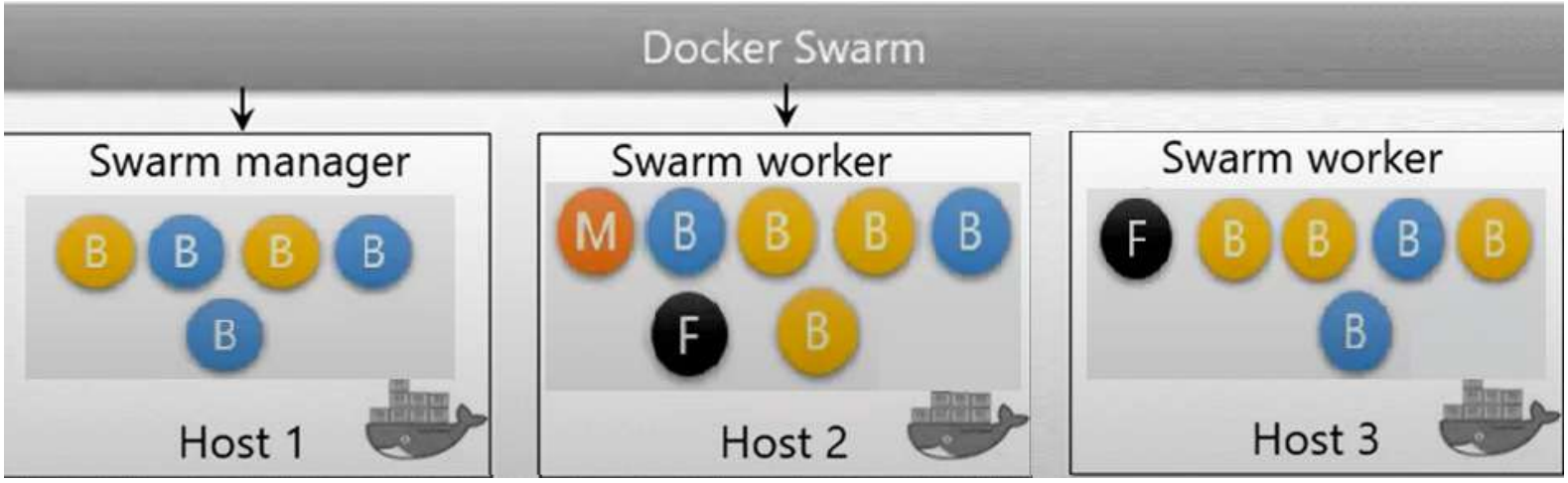


Orchestration des conteneurs

Déployer la mise à jour du service front

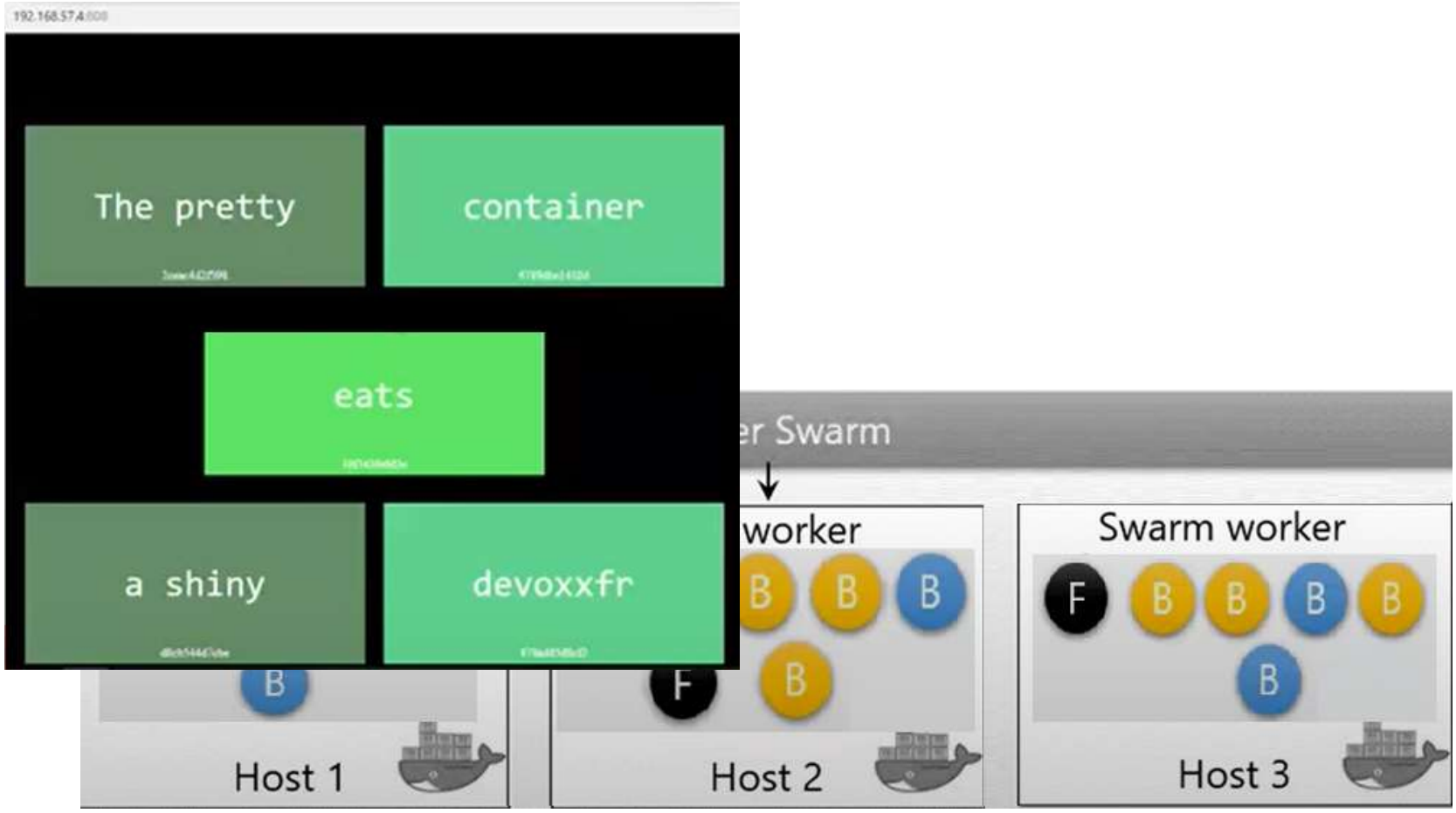
```
$ sudo docker service update --image vdemeester/exquisite-web:v2 front

front
overall progress: 2 out of 2 tasks
1/2: running
2/2: running
verify: Service converged
```



Conteneurisation avec Docker

Déployer la mise à jour du service front

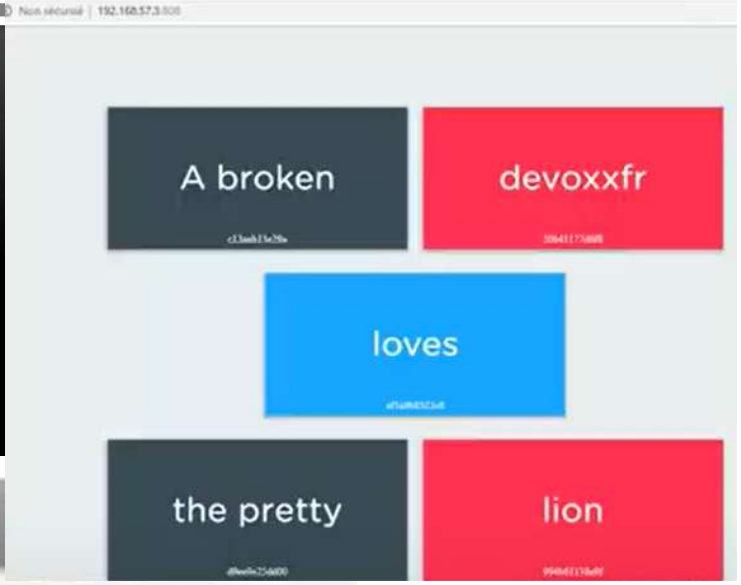


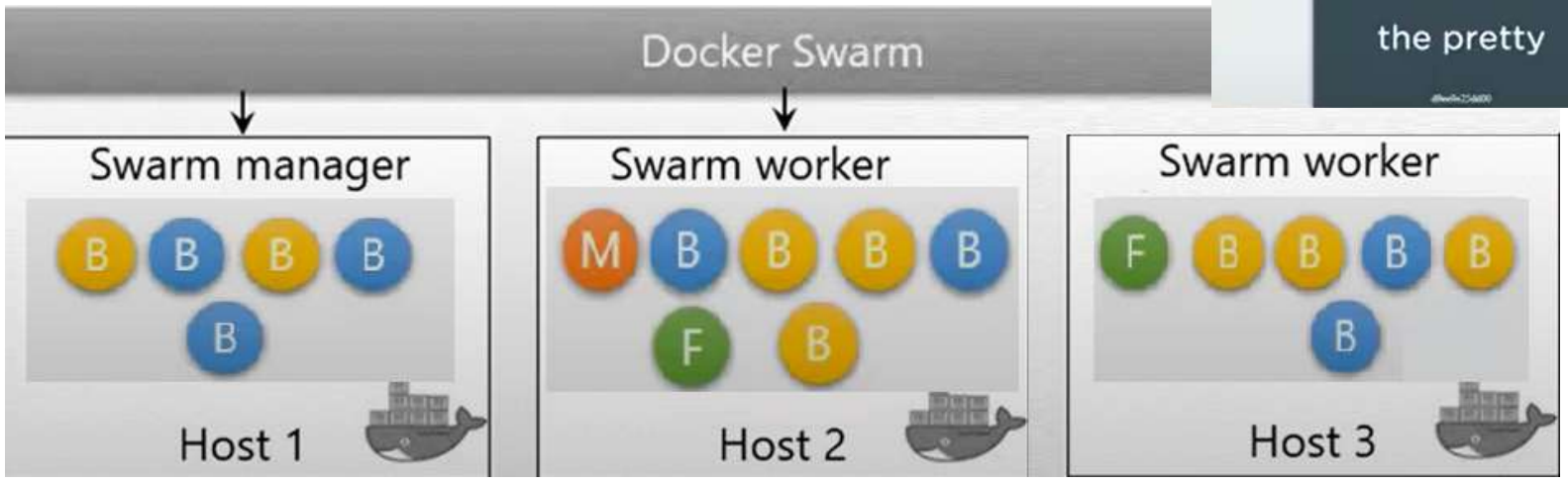
Orchestration des conteneurs

Déployer la mise à jour du service front: Rollback

```
$ sudo docker service update --rollback front

front
rollback: manually requested rollback
overall progress: rolling back update: 2 out of 2 tasks
1/2: running
2/2: running
verify: Service converged
```



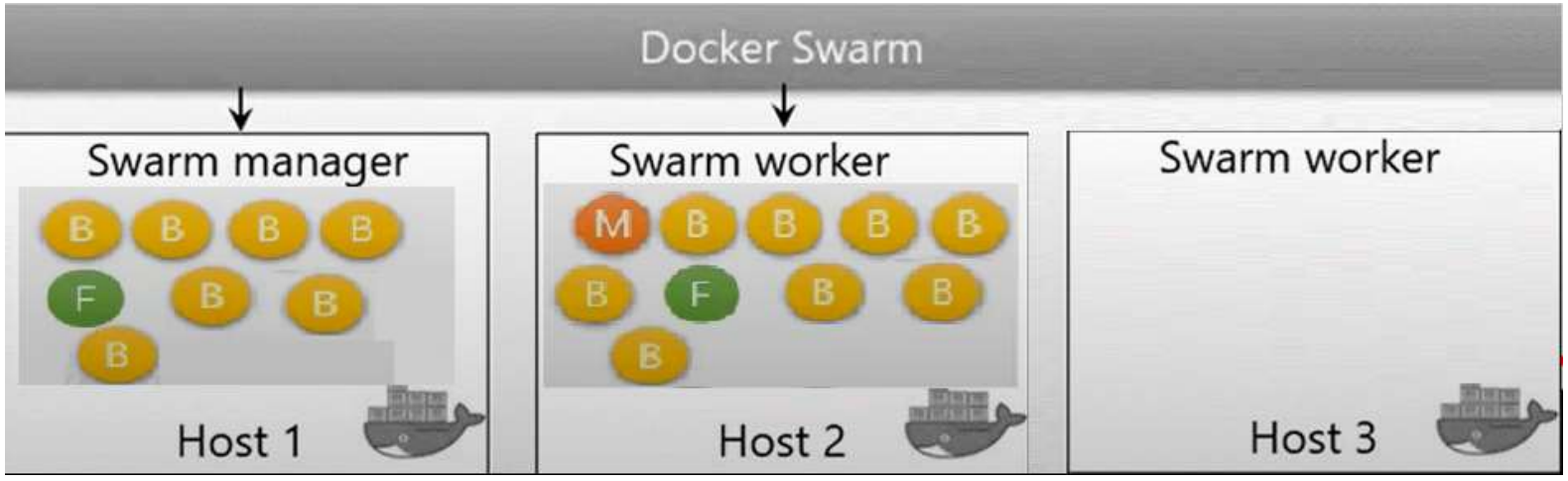


Conteneurisation avec Docker

Suspendre un nœud worker

```
$ sudo docker  node update --availability drain host3
host3
```

Toutes les instances des services déployées dans le nœud Host 3 vont migrer vers les autres nœuds du cluster



Orchestration des conteneurs

Conditionner le démarrage de services

```
$ sudo docker service create --name svc1 --constraint engine.label==ssd nginx
l6e79p5yoh66242t9s684nfhi
overall progress: 0 out of 1 tasks
overall progress: 0 out of 1 tasks
1/1: no suitable node (scheduling constraints not satisfied on 2 nodes; 1 node …
Operation continuing in background.
Use `docker service ps l6e79p5yoh66242t9s684nfhi` to check progress.
youssfi@host1:~$ sudo docker service create --mode global alpine ping 8.8.8.8
bnszchxi5kmj09sf4gfm2qq9u
overall progress: 2 out of 2 tasks
tw9is2zomef5: running
ssslatiguj2k: running
verify: Service converged
```

